

QCM : le pattern Décorateur

8 questions, une seule bonne réponse par question.

Version interactive : [lien à compléter](#)

Question 1. À quelle famille de patterns du GoF appartient le Décorateur ?

- A. Création
- B. Structure
- C. Comportement
- D. Architecture

Question 2. Quelle phrase décrit le mieux l'intention du Décorateur ?

- A. Garantir qu'une classe n'a qu'une seule instance
- B. Convertir l'interface d'une classe en une autre interface attendue par le client
- C. Ajouter dynamiquement des responsabilités à un objet sans modifier sa classe
- D. Choisir un algorithme au moment de l'exécution

Question 3. Dans le diagramme de classes du pattern, la classe Decorateur...

- A. implémente Composant et garde une référence vers un Composant
- B. hérite de ComposantConcret
- C. contient une liste de Composants enfants
- D. ne dépend d'aucune abstraction

Question 4. Une épée fait 12 dégâts. L'enchantement Feu ajoute 5 dégâts, Aiguisee multiplie les dégâts par 2. Que renvoie `new Feu(new Aiguisee(new Epee())).degats()` ?

- A. 34
- B. 22
- C. 17
- D. 29

Question 5. Pour proposer des oignons frits, on crée une classe OignonsFrits sans modifier aucune classe existante. Quel principe SOLID cela illustre-t-il surtout ?

- A. Substitution de Liskov
- B. Ouvert / fermé
- C. Ségrégation des interfaces
- D. Inversion des dépendances

Question 6. Lequel de ces morceaux de code Java utilise le pattern Décorateur ?

- A. `Math.max(a, b)`
- B. `Integer.parseInt("42")`
- C. `new BufferedReader(new FileReader("notes.txt"))`
- D. `String.valueOf(3.14)`

Question 7. Parmi ces affirmations, laquelle est une vraie limite du Décorateur ?

- A. L'ordre dans lequel on empile les décorateurs peut changer le résultat
- B. Il faut modifier le composant de base à chaque nouvelle option
- C. Un objet ne peut être décoré qu'une seule fois
- D. Le client doit connaître chaque décorateur concret pour appeler une méthode

Question 8. Pourquoi peut-on passer `new Cheddar(new TacosPoulet())` à une méthode qui attend un `Tacos` ?

- A. Parce que Cheddar hérite de TacosPoulet
- B. Parce que Java convertit automatiquement les objets
- C. Parce que Cheddar est une classe abstraite
- D. Parce que Cheddar implémente l'interface Tacos (via Supplement)